SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

SE2020 - Web Technologies and Mobile

Ms. Dilani Lunugalage

BSc (Hons) in Information Technology

Lecture 08

ReactJS Part 01

Agenda

1. Introduction to Frontend Development
2. Web Foundations (HTML, CSS, JavaScript)
3. DOM & Virtual DOM
4. Introduction to React
5. React Components & JSX
6. Props and State
7. React Component Lifecycle
8. Event Handling in React
9. React Hooks (useState, useEffect)
10. State Management Basics
11. Build Tools (Babel, Webpack, Vite)
12. Setting Up a Modern React Project

What is Frontend Development?

Frontend development is the process of creating the **visual and interactive layer** of a web application that users directly interact with.

It involves:

- Designing **how the application looks**
- Making it **interactive**
- Ensuring it is **responsive** and **accessible**
- Creating a smooth **user experience**

Runs entirely on:

- **Web browsers** (Chrome, Firefox, Safari, Edge)

Role of a Frontend Developer

A frontend developer focuses on:

- ✓ Building user interfaces
- ✓ Ensuring mobile responsiveness
- ✓ Implementing animations & transitions
- ✓ Integrating with backend APIs
- ✓ Managing browser compatibility
- ✓ Optimizing performance
- ✓ Enhancing accessibility (WCAG standards)

Example tasks:

- ▷ Creating navigation menus
- ▷ Building forms & input validations
- ▷ Rendering data from backend
- ▷ Building SPA components (React, Angular, Vue)

Why Frontend Development is Important

Frontend is the **first impression** *of any application.*

Key reasons:

- Determines **usability & user satisfaction**
- Impacts **user engagement**
- Influences **conversion rate** for business applications
- Affects **brand identity & visual communication**

Technical benefits:

- Efficient UI $\rightarrow$ Less server load
- Enhances app responsiveness
- Enables dynamic, real-time updates
- Modern applications like Facebook, Gmail, YouTube rely heavily on powerful frontend engineering.

Frontend vs Backend

Frontend (Client-Side)
Runs inside the browser
Handles UI, animations, events, forms
Focus on:

- Layout design
- Responsiveness
- User interaction
- Accessibility

Backend (Server-Side)
Runs on servers
Handles:

- Business logic
- Data processing
- Authentication
- APIs & databases

Example in real world:
Frontend = what you see on YouTube (videos, buttons, search box)
Backend = storing videos, user accounts, recommendations

Frontend Has Two Broad Areas

1. UI/UX Design

Focus on:

- Visual layout
- Color theory
- Typography
- User navigation
- Accessibility (A11Y)
- Human–Computer Interaction principles

Tools used:

- Figma
- Adobe XD
- Canva

2. Frontend Engineering

Focus on:

- HTML, CSS, JavaScript
- Frameworks (React, Angular, Vue)
- Optimization & performance
- Build tools (Vite, Webpack)
- Testing tools (Jest, React Testing Library)
- DevOps (CI/CD for frontend)

The Frontend Technology Stack

1. **Core Languages**
   - HTML5
   - CSS3
   - JavaScript (ES6+)
2. **Frameworks & Libraries**
   - React
   - Angular
   - Vue
3. **Styling Tools**
   - Tailwind CSS
   - Bootstrap
   - Material UI
   - Sass / Less
4. **Build Tools**
   - Vite
   - Webpack
   - Parcel
5. **Version Control**
   - Git
   - GitHub / GitLab / Bitbucket

Types of Frontend Applications

1. **Static Websites**
   - HTML + CSS
   - No dynamic content
   - *Examples:* Personal portfolio, company introduction page
2. **Dynamic Websites**
   - JS-based interactivity
   - Form submissions
   - Displaying dynamic data
   - *Examples:* News websites, blog systems
3. **Single Page Applications (SPA)**
   - Entire application loads once
   - New content rendered using JavaScript
   - Fast, app-like experience
   - Tools: **React, Vue, Angular**
   - *Examples:*
     - Facebook
     - Gmail
     - Twitter Web App

The Need for Frameworks like React

Traditional frontend development had challenges:

- Hard to manage complex UIs
- DOM updates are slow
- Code becomes difficult to maintain
- No structured component system
- Reusability is limited

React solves these problems with:

- ✓ Components
- ✓ Virtual DOM
- ✓ State management
- ✓ Hooks
- ✓ Declarative programming

Web Foundations Overview

To build any frontend application (including React), you need three core technologies:

HTML → Structure

- Defines the layout and elements of the web page
- (Headings, paragraphs, forms, images, buttons)

CSS → Styling

- Controls the appearance
- (Colors, fonts, spacing, layout, responsiveness)

JavaScript → Behavior

- Makes the website interactive
- (Events, logic, API calls, dynamic updates)

HTML (Structure)

HTML provides the **skeleton** of a webpage.

Key points:

- Uses **tags** to organize content
- Creates headings, paragraphs, images, lists
- Semantic HTML improves SEO and accessibility

Example:

```
<h1>Welcome</h1> <p>This is my website</p>
```

- In React, you will write HTML-like syntax using **JSX**.

CSS (Styling & Layout)

CSS controls how HTML elements **look and adjust** on different screen sizes.

What CSS does:

- Colors, fonts, spacing
- Layout using Flexbox & Grid
- Animations & transitions
- Responsive design for mobile/tablets

Example:

```
button background: blue; color: white;
```

React integrates well with CSS frameworks like **Tailwind, MUI, Bootstrap**.

JavaScript (Interactivity & Logic)

JavaScript brings the page to life.

What JS enables:

- Click events & user interactions
- Form validation
- API calls (fetching data from backend)
- Changing the DOM dynamically
- Running logic inside the browser

Example:

```
 document.getElementById("btn").onclick = () =>
alert("Clicked!");
```

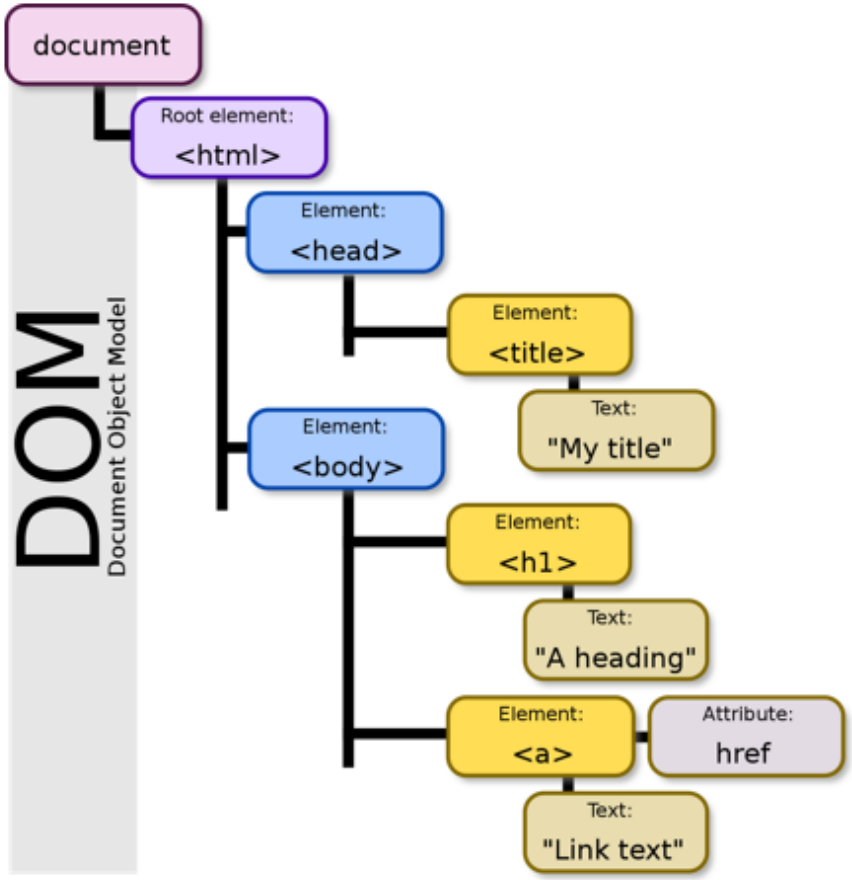
- React is built on JavaScript and makes UI updates easier and faster using **components** & **Virtual DOM**.

What is the DOM? (Document Object Model)

The **DOM** is the browser's internal representation of your webpage.

Key characteristics:

- Represents HTML as a **tree structure**
- Each element = a **node**
- Allows JavaScript to **read, modify, create, or delete** elements
- React ultimately interacts with the DOM through JSX



Source: `https://en.wikipedia.org/wiki/Document_Object_Model`

Why DOM Exists?

The browser converts HTML $\rightarrow$ DOM so it can:

- ✓ **Render content visually**
- ✓ **Process styling (CSS)**
- ✓ **Handle events (click, scroll, input)**
- ✓ **Update content when JS modifies it**

This makes DOM essential for dynamic pages.

DOM Manipulation with JavaScript (Traditional Way)

JavaScript directly modifies the DOM:

Example:

```
document.getElementById("title").innerText = "Updated Title";
```

Problems in complex UIs:

- Repeated DOM access is **slow**
- Browser has to re-calculate layout
- UI gets **janky** in large applications
- Hard to maintain when UI grows
- Risk of inconsistent UI state

What is the Virtual DOM? (React's Secret Weapon)

The **Virtual DOM (VDOM)** is a **lightweight JavaScript representation** of the actual DOM.

Key Points:

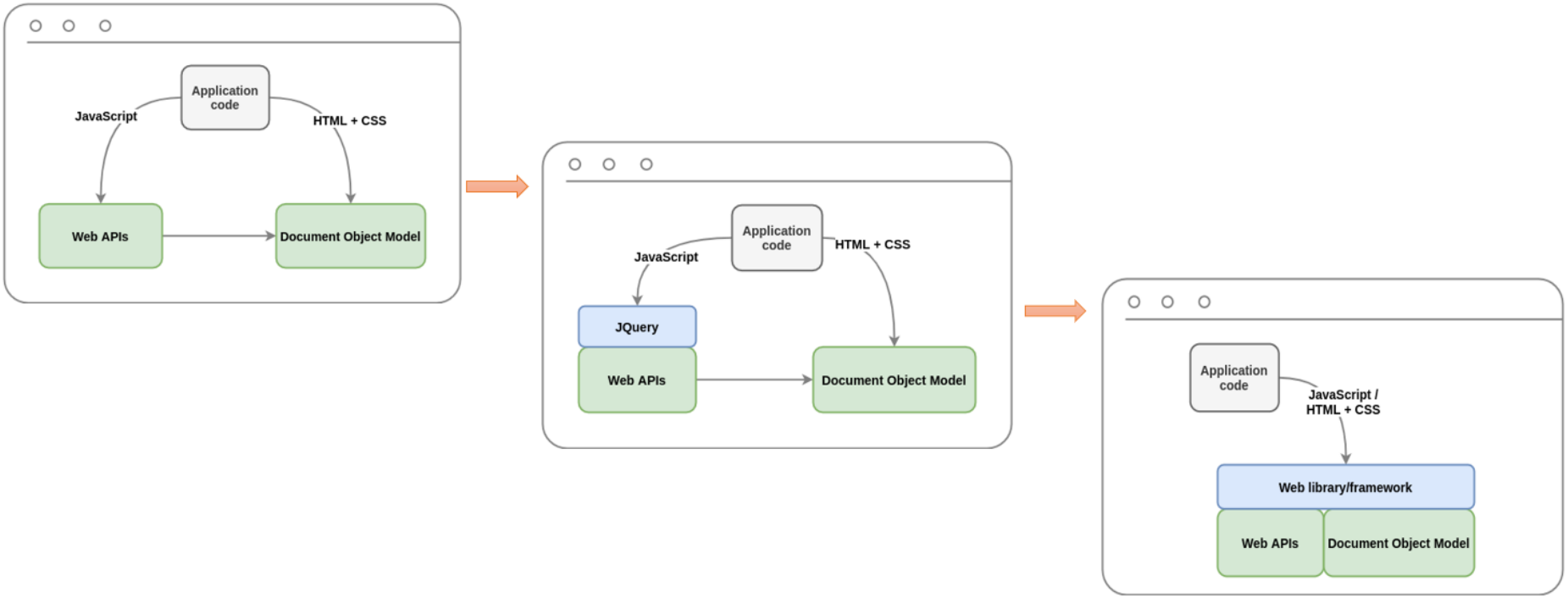
- React keeps a copy of the UI in memory
- Updating the Virtual DOM is **faster** than updating the real DOM
- React uses this to decide **when and what** to update

Diagram:
Actual DOM (slow) $\rightarrow$ *Virtual DOM (lightweight copy)* $\rightarrow$ *React Components (JSX)*

DOM vs Virtual DOM

Feature	DOM	Virtual DOM
Nature	Real browser structure	JS object representation
Update Speed	Slow	Fast
Re-render	Whole page or large sections	Only changed nodes
Efficiency	Low for large UIs	Very high
Used By	Browser	React

Evolution of Frontend Development



Source: `https://www.epineda.net/the-evolution-of-front-end-development/amp/`

ReactJS

Introduction to React

React is a **JavaScript library** used to build modern, interactive user interfaces.

Key characteristics:

- Developed and maintained by **Meta (Facebook)**.
- Used for building **Single Page Applications (SPAs)**.
- Represents the **View** layer in the MVC architecture.
- Ideal for large applications that update content **without reloading the entire page**.
- Supports multiple platforms:
  - **ReactJS** for web
  - **React Native** for mobile (Android/iOS)
  - **React 360 / React VR** for VR/AR
- Focuses on **speed, simplicity, and scalability**.
- Examples: Facebook, Instagram, WhatsApp Web, Netflix UI

React is widely adopted in industry due to its simplicity and performance.

Why React?

React solves modern UI challenges through:

- ✓ **Fast Rendering (Virtual DOM)**
  Only updates changed parts of UI → very efficient.
- ✓ **Component-Based Architecture**
  Build UI like LEGO blocks → reusable, maintainable.
- ✓ **Declarative UI**
  You describe *what* UI should look like, React handles *how* to update it.
- ✓ **One-Way Data Flow**
  Predictable behavior, easy debugging.
- ✓ **Cross-platform**
  React.js → Web — React Native → Mobile + React VR → VR environments

JSX (React Syntax)

JSX = **JavaScript XML**

Features:

- Allows writing HTML-like syntax in JavaScript.
- Easy to read and write.
- Understands dynamic JavaScript expressions.
- Compiled by *Babel* into pure *JavaScript*.
- Prevents XSS
- Supports expressions inside {}
- Easier maintenance

Example:

```
const element = <h1>Welcome to React</h1>;
```

JSX is the foundation of building React UI.

JSX (React Syntax)

JSX Rules

- Must return only **one parent element**

```
return ( <div> <h1>Hello</h1> </div> )
```

- Use **className** instead of class
- JS expressions inside {}
- Must close all tags
- No "if" inside JSX → use ternary

How React Components Render

1. Component returns **JSX**
2. *Babel* converts JSX → JavaScript
3. React creates Virtual DOM
4. React compares VDOM with old VDOM
5. Only the changed UI updates in real DOM

React File Structure

- **/src** → main development files
- **/components** → reusable UI pieces
- **App.jsx** → main app
- **index.jsx** → mounts app to DOM

React Components

Components are the **building blocks** of every React UI.

Why Components?

- Make UI **modular** and **reusable**
- Each component has its own structure, logic, and styling
- Simplifies large application development

Types of Components:

- **Functional Components** (modern standard)
- **Class Components** (older React)

```
App
 ├── Navbar
 ├── Sidebar
 ├── Dashboard
 │      ├── Card
 │      ├── Chart
 │      └── SummaryBox
 └── Footer
```

Example UI Breakdown:

Everything you see on screen can be broken into components.

Types of Components

1. Functional Components
(Modern React, Recommended)

- Simple JS functions
- Use Hooks like useState, useEffect
- Easy to read & maintain

Example:

```jsx
function Hello() {
  return <h1>Hello React!</h1>;
}
```

2. Class Components (Older React)

- Uses class syntax
- Has lifecycle methods
- Still important for legacy projects

Example:

```jsx
class Hello extends React.Component {
  render() {
    return <h1>Hello React!</h1>;
  }
}
```

What Are Props?

Props = "properties"

They allow you to **pass data from parent → child components**.

Key Characteristics

- **Read-only** (immutable)
- Cannot be changed by the child
- Make components **reusable**
- Passed similar to HTML attributes

Example Use Cases

- Displaying dynamic content
- Customizing components
- Passing callback functions

Props

Example

Parent Component

```
<App>
 <Welcome name="WMT" />
</App>
```

Child Component

```
function Welcome(props) {
 return <h1>Hello, {props.name}</h1>;
}
```

Output:

Hello, WMT

- Shows data passing
- Shows JSX interpolation

Output: Hello, WMT

Why Props Are Important

- Props help you build **configurable components**, e.g.:

```
<Button text="Save" color="green" />
<Button text="Cancel" color="red" />
```

- Same component, different behavior → reusable UI.

What is State?

State = data that changes over time inside a component.

React updates UI whenever the state changes.

✓ **Examples of state:**

- ☐ Toggle button
- ☐ Dropdown open/close
- ☐ Logged-in user
- ☐ Counter value
- ☐ API response data
- ☐ Form inputs

State allows React components to be **interactive**.

State

Example

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}
```

- **useState(0)** initializes
- **setCount** updates
- **UI re-renders automatically**

State

Important Rules of State

- ✓ **Rule 1: Never modify state directly**
  - × `count = count + 1`
  - ✓ `setCount(count + 1)`
- ✓ **Rule 2: State updates are asynchronous**
- ✓ **Rule 3: Every state change triggers a re-render**
- ✓ **Rule 4: Put minimal required data in state**
  Avoid storing values that can be calculated.

Passing Functions as Props (Callback Pattern)

Child → Parent communication is done by passing functions as props.

Parent

```
function Parent() {
  function handleClick() {
    alert("Child clicked!");
  }

  return <Child onButtonClick={handleClick} />;
}
```

Child

```
function Child(props) {
  return <button onClick={props.onButtonClick}>Click</button>;
}
```

Component Lifecycle

React components go through **3 main phases**:

- **Mounting**
  - Component is created & inserted into the DOM.
- **Updating**
  - Triggered when **state or props** change.
- **Unmounting**
  - Component is removed from the DOM.

React provides lifecycle **methods (Class Components)** and **hooks (Functional Components)** to run code during these stages.

Mounting Phase (Class Components)

These methods run when the component FIRST appears onscreen.

1. **constructor(props)**
   - Initialize state
   - Bind event handlers
   - Runs before component renders
2. **render()**
   - Returns JSX
   - Must be pure, no side effects
3. **componentDidMount()**
   - Runs AFTER component is added to DOM
   - Good for:
     - Fetching API data
     - Subscribing to events
     - Initializing timers

Example Use:

```
componentDidMount() {
  fetchUserData();
}
```

Updating Phase (Class Components)

Triggered when props or state change.

1. **shouldComponentUpdate(nextProps, nextState)**
   - Controls re-rendering
   - Return false to avoid unnecessary UI updates (performance optimization)
2. **render()** - Re-renders UI with updated state/props
3. **getSnapshotBeforeUpdate(prevProps, prevState)**
   - Captures information from DOM before update
   - Useful for scroll position, animation, etc.
   - Returned value goes into componentDidUpdate()
4. **componentDidUpdate(prevProps, prevState, snapshot)**
   - Runs after component updates
   - Good for:
     - Updating DOM
     - Making API calls based on new data
     - Acting on the snapshot

Unmounting Phase (Class Components)

When component is about to be removed from the DOM:

componentWillUnmount()
Used for **cleanup**:

- Remove event listeners
- Stop timers
- Cancel network requests
- Clean up subscriptions

Example:

```
componentWillUnmount() {
  clearInterval(this.timer);
}
```

Functional Components Lifecycle (using useEffect)

Functional components replace lifecycle methods with **useEffect**:

- ✓ **Mount + Update**
  useEffect(() => console.log("Runs on every render"););
- ✓ **Mount Only**
  useEffect(() => console.log("Runs once on mount");,[]);
- ✓ **Unmount (Cleanup)**
  useEffect (() => return () => console.log("Component unmounted");;,[]);
- ✓ **Update Only (watch specific state)**
  useEffect(() => console.log("Runs when count changes");,[count]);

React Component Lifecycle

componentDidMount():

- This method is called immediately after a component is mounted in the DOM. It is often used for initializing the component state or fetching data from an API.

componentWillUnmount():

- This method is called immediately before a component is unmounted from the DOM. It is often used for cleaning up any resources or event listeners associated with the component.

shouldComponentUpdate(nextProps, nextState):

- This method is called before a component is re-rendered. It returns a boolean value indicating whether the component should update or not, based on the next props and state values.

getDerivedStateFromProps(props, state):

- This is a static method that is called before rendering a component, both on the initial mount and on subsequent updates. It returns an object to update the component state based on the new props.

getSnapshotBeforeUpdate(prevProps, prevState):

- This method is called right before the most recent render output is committed to the DOM. It allows you to capture information about the DOM before it is changed, such as the scroll position, and return it as an object.

componentDidUpdate(prevProps, prevState, snapshot):

- This method is called immediately after a component is updated and re-rendered. It is often used for updating the component state or making API calls based on the new props or state.

All the above ones can be represented from useEffect() hook in React Functional Components

Mapping Class Methods to useEffect

Class Method	Functional Equivalent (Hooks)
constructor()	useState()
componentDidMount()	useEffect(() => {}, [])
shouldComponentUpdate()	React.memo(), useMemo, useCallback
render()	return() in function component
getSnapshotBeforeUpdate()	useLayoutEffect()
componentDidUpdate()	useEffect(() => {}, [dependencies])
componentWillUnmount()	cleanup function in useEffect()

What Are Event Handlers?

Event handlers are **functions** that run when the user interacts with the UI.

React responds to events such as:

- Clicking a button
- Typing in an input
- Submitting a form
- Moving the mouse
- Pressing a key

React uses a **Synthetic Event System** which:

✓ Normalizes events across browsers

✓ Improves performance with event delegation

Event Handling in React

Event Handling Syntax in React

React uses:

- **camelCase event names**
- **JS functions** instead of strings

Example:

```
<button onClick={handleClick}>Click Me</button>
```

Writing an Event Handler Function

```
function Button() {
  function handleClick() {
    alert("Button clicked!");
  }

  return <button onClick={handleClick}>Click</button>;
}
```

Important:

- Handler can have any name
- You can define it inside the component
- React binds events automatically (no addEventListener)

Event Handling in React

Passing Arguments to Event Handlers

Use an arrow function when passing parameters:

```
<button onClick={() => handleClick(5)}>Increase by 5</button>
```

Handler:

```
function handleClick(amount) {
  setCount(count + amount);
}
```

Useful for:

- Updating specific items
- Passing IDs
- Dynamic actions

Event Handling in React

Event Object

React event handlers receive an event object (SyntheticEvent).

```
function handleInput(e) {
  console.log(e.target.value);
}

<input type="text" onChange={handleInput} />
```

The event object contains:

- e.target → element
- e.target.value → typed text
- e.clientX / e.clientY → mouse position
- e.key → key pressed

Common React Events

Mouse Events

- onClick
- onDoubleClick
- onMouseEnter
- onMouseLeave

Form Events

- onChange
- onSubmit
- onInput

Keyboard Events

- onKeyDown
- onKeyPress
- onKeyUp

Clipboard Events

- onCopy
- onPaste

Event Handling in React

Handling Form Submissions

React prevents page reload (default form behavior):

```
function handleSubmit(e) {
  e.preventDefault();
  console.log("Form submitted");
}

<form onSubmit={handleSubmit}>
  <input />
  <button type="submit">Submit</button>
</form>
```

e.preventDefault() is essential in React forms.

Inline Event Handlers vs Named Functions

Named handler (recommended):

```
<button onClick={handleClick}>Save</button>
```

Inline function:

```
<button onClick={() => alert("Saved")}>Save</button>
```

Best practice:

Use **named handlers** for clarity and performance.

What Are React Hooks?

Hooks are special functions that let functional components use React features.

WHY Hooks?

- Functional components couldn't use state or lifecycle before Hooks
- Easier than class components
- Cleaner & more reusable logic
- No need for `this` keyword
- Encourages functional programming

WHAT Hooks Replace:

- `this.state` → **useState**
- `componentDidMount` / `Update` / `Unmount` → **useEffect**
- Context Consumer → **useContext**
- Redux-like reducers → **useReducer**

React Hooks

Rule of Hooks

Hooks work only if you follow 2 strict rules:

1. **Call Hooks at the TOP level**
   - Always call at the top of the component
2. **Call Hooks only inside React Components or Custom Hooks**
   - Inside functional components
   - Inside custom hooks

Reason: React needs to track hook order.

The most common Hooks are

- useState
- useEffect
- useContext
- useReducer

React Hooks

useState Hook

useState allows components to store & update values.

Syntax:

```
const [value, setValue] = useState(initialValue);
```

Example

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

Key Points:

- Updating state re-renders component
- Use **functional updates** when state depends on previous value

```
setCount(prev => prev + 1);
```

useEffect Hook

useEffect runs **side effects** → tasks that happen *outside* the UI.

Examples of Side Effects:

- Fetching API data
- Subscribing to events
- Timers
- Manipulating DOM
- Saving to localStorage

Syntax Variants:

- **Run on every render**

  useEffect(() => {});

- **Run only on mount (componentDidMount)**

  useEffect(() => {}, []);

- **Run when dependencies change**

  useEffect(() => {}, [count]);

- **Cleanup on unmount**

  useEffect(() => { return () => console.log("cleanup"); }, []);

React Hooks

useContext

useContext → Avoids "props drilling"

Without context:

App → Dashboard → Sidebar → UserProfile

With context, pass data **directly to any child**, no matter how deep.

Example:

const value = useContext(MyContext);

Used for:

- Theme
- User session
- Cart

useReducer

useReducer → Complex state logic

Used instead of multiple useState updates.

Example:

```
function reducer(state, action) {
  switch(action.type) {
    case "add": return state + 1;
    default: return state;
  }
}

const [state, dispatch] = useReducer(reducer, 0);
```

useReducer is:

- Predictable
- Scalable
- Similar to Redux

What Is State?

State = component's **current data**, which determines **how the UI looks**.

Examples:

- Text inside an input
- Whether a modal is open
- Selected list item
- API data (users, products)
- Logged-in user info

Why State Matters:
Without state, your UI would be static and never change.

Managing State | Reacting to Inputs

Why React State Matters

- As your app grows, you must carefully organize **where state lives** and **how it flows** between components.
- Redundant or duplicate state often leads to bugs and inconsistent UI.
- React encourages a **declarative mindset**:
  - Instead of updating individual DOM elements manually
  - You describe *states* your component can be in
  - React updates UI when state changes

Designer-Like Thinking

Think about UI as a set of **states** and **transitions** based on user input (clicks, typing, selections).

Managing State | State Structure

Structuring state well can make a difference between a component that is pleasant to modify and debug, and one that is a constant source of bugs.

Key Principles:

- **Group related state.** If you always update two or more state variables at the same time, consider merging them into a single state variable.
- **Avoid contradictions in state.** When the state is structured in a way that several pieces of state may contradict and "disagree" with each other, you leave room for mistakes. Try to avoid this.
- **Avoid redundant state.** If you can calculate some information from the component's props or its existing state variables during rendering, you should not put that information into that component's state.
- **Avoid duplication in state.** When the same data is duplicated between multiple state variables, or within nested objects, it is difficult to keep them in sync. Reduce duplication when you can.
- **Avoid deeply nested state.** Deeply hierarchical state is not very convenient to update. When possible, prefer to structure state in a flat way.

Managing State | State Sharing

Sometimes two or more components need to share the same state.

How to Handle This:

1. Move the shared state to the **closest parent** component.
2. Pass the state down via **props**.
3. Pass update functions down via **props**.

This Technique is Called: **-> Lifting State Up**

When to Lift State?

- Two components must be synchronized
- Parent must coordinate data
- A change in one component should update another

This is one of the most common patterns you'll use in real React projects.

```
function Parent() {
  const [value, setValue] = useState("");

  return (
    <>
      <InputBox value={value} onChange={setValue} />
      <PreviewBox text={value} />
    </>
  );
}
```

- Input and preview synced
- Controlled through parent
- No duplicate state

Managing State | Preserving & Resetting State

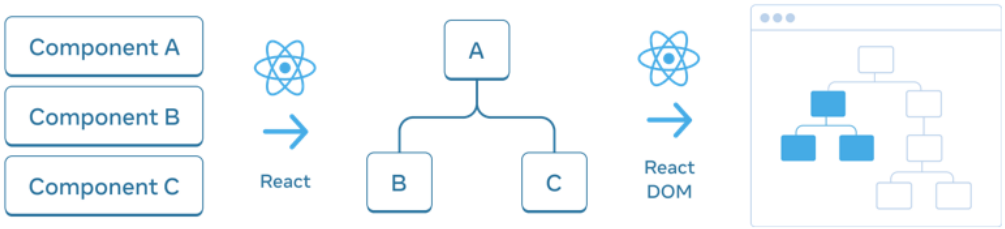
React decides whether to **preserve or reset** component state based on its **position in the UI tree**.

✓ **State is Preserved When:**

- Component stays in the same position
- The component's key does not change

X **State is Reset When:**

- Component is conditionally removed from DOM
- Its key changes
- Its position changes in the component tree



Example:

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

Switching components resets the state of whichever is unmounted.

Understanding this helps avoid unintended data loss and form resets.

Managing State | Using Reducer

When state updates become complicated:

- Multiple state variables
- Many corresponding event handlers
- Logic spread across multiple places

This can get overwhelming.

Solution: useReducer

A reducer:

- Centralizes all state update logic
- Makes updates predictable
- Similar concept to Redux

Reducer Example:

```
function reducer(state, action) {
  switch (action.type) {
    case "add":
      return { count: state.count + 1 };
    case "reset":
      return { count: 0 };
    default:
      return state;
  }
}
```

Reducers improve clarity for large or complex components.

Managing State | With Context

- Usually, you will pass information from a parent component to a child component via props.
- But passing props can become verbose and inconvenient if you have to pass them through many components in the middle, or if many components in your app need the same information.
- Context lets the parent component make some information available to any component in the tree below it—no matter how deep—without passing it explicitly through props.

Context Solves This:

- Provides global values
- Accessible anywhere in the component tree
- No need to pass props manually

Great for:

- Authentication (logged-in user)
- Theme ,Language selection
- Cart state
- Global app settings

```
App
└── Page
    └── Sidebar
        └── Menu
            └── UserProfile
```

Be careful:

Overusing context causes unnecessary re-renders → Use wisely.

Build Tools in Modern Web Development

Why We Need Build Tools

Modern web applications rely on:

- JSX
- ES6+ features (let/const, classes, async/await)
- Modules (import/export)
- TypeScript
- CSS preprocessors
- Images, fonts, assets
- Large component-based architectures

Build tools help developers by:

- **Transpiling** code → browser-compatible
- **Bundling** files → fewer HTTP requests
- **Optimizing** output → smaller & faster apps
- **Running dev servers** with hot reload
- **Improving developer experience**

Without them: React/Vue/Svelte apps would not run reliably in all browsers.

Babel

- Babel is a JavaScript compiler that allows you to write modern JavaScript code and transform it into code that can run in older browsers or environments.
- Babel can transform modern JavaScript features like arrow functions, template literals, and de-structuring into equivalent code that is compatible with older browsers.
- Babel uses plugins to transform specific features of JavaScript. There are many plugins available for Babel, and you can choose which ones to use based on your needs.
- Babel can also transform code written in other languages that compile to JavaScript, like TypeScript and JSX.
- Babel is open-source software and is maintained by a community of developers. It is a widely used tool in the JavaScript ecosystem and is supported by many popular libraries and frameworks.
- Babel supports many advanced features of JavaScript, such as async/await, class properties, and decorators, allowing you to use the latest language features even in older environments.

Bundling

Bundling is the process of **combining multiple files and modules into a single optimized file** that can be loaded by the browser efficiently.

Why Bundling Is Needed

- Modern applications use many JavaScript modules, components, images, and styles.
- Browsers handle **fewer, larger files better** than many small files.
- Bundling reduces the number of **HTTP requests**, improving performance.
- Ensures all dependencies are loaded in the correct order.

What Bundlers Do

Bundling tools analyze the **dependency graph**:

Then they merge everything into one or few production files (e.g., `bundle.js`).

Bundling

Bundling + Optimization Techniques

Most bundlers also apply:

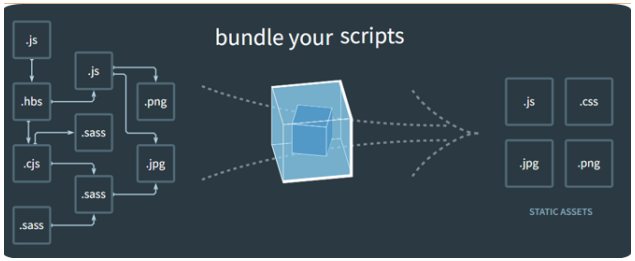
- **Minification** → removes spaces, comments, shortens variable names
- **Tree Shaking** → removes unused code
- **Code Splitting** → loads code only when necessary
- **Caching** → faster reloads on subsequent page visits

Why It Matters

Bundling:

- Improves performance
- Reduces latency
- Ensures compatibility
- Allows modular code structure while shipping optimized output

Webpack



Webpack

Webpack is the most widely used **module bundler** in modern JavaScript development.

What Webpack Does

- Bundles JavaScript modules into one or more optimized files
- Processes assets like CSS, images, fonts, JSON, and more
- Builds a **dependency graph** to understand how modules depend on each other
- Generates a final **production-ready bundle**

How Webpack Works

1. Webpack starts from your **entry file** (e.g., index.js)
2. It analyzes all imports and builds a **dependency graph**
3. Each file is processed using **loaders**
4. Webpack uses **plugins** to optimize the bundle
5. The final output is stored in a dist/ folder (e.g., bundle.js)

Webpack

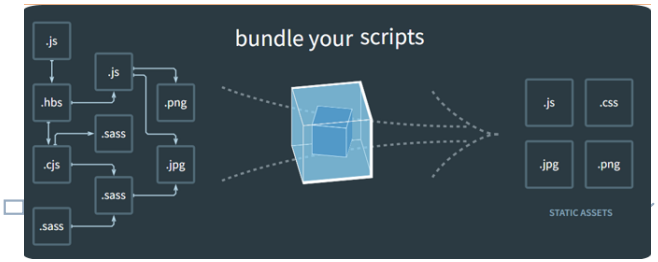
Key Features

- **Loaders:** Convert non-JS files (CSS, images, JSX, TypeScript) into modules
- **Plugins:** Optimize output (minification, compression, environment variables)
- **Code Splitting:** Split bundles to load only what's needed
- **Tree Shaking:** Remove unused code to reduce file size
- **Hot Module Replacement (HMR):** Update modules without full refresh
- **Dev Server:** Embedded server for live preview and auto reload

Why Webpack Is Important

- Works with all modern frameworks: React, Angular, Vue
- Highly customizable for complex enterprise applications
- Mature plugin ecosystem with thousands of extensions
- Handles complex workflows like SCSS, TypeScript, image optimization, etc.

Official Website https://webpack.js.org/



Parcel

Parcel

Parcel is a **zero-configuration web application bundler** that aims to provide an extremely fast, simple, and developer-friendly build experience.

Why Parcel Is Popular

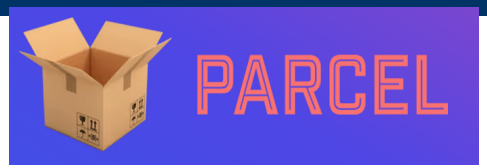
- **Zero Configuration:** Works out of the box—no Webpack-style config files required
- **Fast Build Times:** Uses multi-core processing to bundle assets in parallel
- **Simple Setup:** Perfect for small/medium applications and beginners
- **Automatic Transformations:** Detects file types and applies the right transformations automatically

Parcel

Parcel Features

- Bundles **JavaScript, CSS, HTML, images, fonts, JSON**, and more
- Automatically transpiles JS (via Babel) & TypeScript without extra setup
- Built-in support for **JSX**, **React**, and **modern JavaScript** features
- Built-in **development server** with hot reloading
- Intelligent **caching system** → faster rebuilds
- Built-in optimizations for production:
  - ✓ Minification
  - ✓ Tree shaking
  - ✓ Image compression
  - ✓ Code splitting

Parcel



Advanced Capabilities

- Supports **multiple entry points** (useful for multi-page apps)
- Friendly error messages & diagnostic overlays
- Integrates easily with frameworks like **React, Vue, Svelte**
- Automatically detects installed dependencies & adjusts pipeline

When to Use Parcel

Use Parcel when:

- You want a fast and simple tool
- You don't want to maintain a long Webpack config
- You're building a small or medium-sized project
- You want bundling + dev server + JSX/TS support out of the box

Official Website `https://parceljs.org/`

Vite

Vite

Vite is a **next-generation build tool and development server** designed for modern JavaScript frameworks like React, Vue, and Svelte.

Why Vite Was Created

Traditional bundlers (Webpack, Parcel) become slow because:

- They must bundle the entire project *before* serving it
- Rebuilding on file change is expensive

Vite solves this using **native ES modules** and an ultra-fast dev server.

Vite

Key Features of Vite

- **Super Fast Startup**

  Vite launches instantly, regardless of project size.
- **Native ES Module Support**

  Serves source files directly to the browser using ES modules (no pre-bundling).
- **Lightning-Fast HMR (Hot Module Replacement)**

  Only the changed modules are updated—no full page reload.
- **Optimized Production Build**

  Uses **Rollup** under the hood to generate efficient production bundles.
- **Minimal Configuration**

  Works out of the box; only customize if needed.

How Vite Works

Development:

- Browser loads modules directly
- No bundling needed → instant startup

Production:

- Vite internally uses **Rollup** to:
  - Bundle
  - Tree-shake
  - Minify
  - Code-split

This produces a highly optimized build.

Vite

Benefits

- 10×–20× faster development experience
- Built-in support for:
  - JSX
  - TypeScript
  - CSS, PostCSS
  - JSON, images, assets
- Plugin system compatible with Rollup plugins
- Great choice for modern frameworks

When to Use Vite

Use Vite when:

- You're building a modern React or Vue application
- You want the fastest dev server
- You want minimal hassle & configuration
- You care about fast HMR and smooth development flow

Vite is now the **official recommended tool** for starting a new React project.

Official Website :- https://vitejs.dev/

Webpack vs Parcel vs Vite

Feature	Webpack	Parcel	Vite
Developer Experience	Moderate → requires config	Easy → minimal setup	Very easy → instant setup
Configuration	Highly configurable but complex	Zero/low configuration	Minimal configuration
Build Time (Development)	Slow for large apps (bundles everything first)	Fast for small/medium apps	Extremely fast (no bundling on dev)
Build Time (Production)	Fast but depends on config	Fast	Very fast (Rollup optimized)
Hot Reloading (HMR)	Built-in but slower	Very fast	Super fast, instant updates
Tree Shaking	Yes	Yes	Yes
Code Splitting	Yes, highly customizable	Yes	Yes (Rollup-powered)
Plugin Ecosystem	Largest & most mature	Growing but smaller	Large (uses Rollup ecosystem)
Learning Curve	Steep	Beginner-friendly	Very beginner-friendly
Community Support	Long-established, huge	Medium-sized	Growing rapidly
Best Use Case	Large enterprise applications	Small to medium projects	Modern React/Vue apps requiring speed
Popularity Trend	Very popular & stable	Popular but less than Webpack	Exploding popularity

Setting Up React

React Official Recommendation

The official React documentation now recommends **using a framework** instead of manually setting up React with bundlers like Webpack or even Create React App.

Why?

- Frameworks solve many real-world problems automatically:
  - *Routing*
  - *Data fetching*
  - *Server-side rendering (SSR)*
  - *Code splitting*
  - *SEO*
  - *File-based routing*
  - *Optimized production builds*

Frameworks give you a complete environment from day one.

Setting Up React

Popular Frameworks for React

1. **Next.js (Most Popular & Officially Recommended)**
   - Full-stack React framework
   - File-based routing
   - SSR, SSG, API routes
   - Excellent performance & SEO
   - Ideal for production-grade apps
2. **Remix**
   - Focused on web fundamentals
   - Optimized data loading & routing
   - Great for building fast, interactive applications
3. **Gatsby**
   - Static site generator
   - Ideal for blogs, documentation, marketing sites
   - Plugin ecosystem and blazing-fast static builds
4. **Expo (for React Native apps)**
   - Build mobile apps using React
   - Uses a single JavaScript codebase for Android + iOS
   - Comes with built-in tools, camera, notifications, assets

Thank You!